

Services et contrats de la cible Rust

ÉTUDE DE DOMAINE

Architecture cible : domaines Rust, contrats et états

Ce découpage représente des responsabilités, pas une obligation de déployer dix-huit microservices dès le premier lot. Plusieurs cores peuvent partager un binaire initial tout en séparant leurs autorisations, données et interfaces.

Responsabilités de contrôle et de données

- Chaque domaine métier possède un core Rust testable sans réseau, entouré d'adaptateurs HTTP/gRPC, catalogue, stockage et ordonnanceur. Les frontends TypeScript et les workloads Python/SQL restent des interfaces et des exécutants. Un tiers est remplaçable derrière le contrat du domaine ; il ne choisit ni droits, ni édition autoritaire, ni checkpoint.
- Les appels de contrôle acceptent des clés d'idempotence et renvoient une opération consultable. Les événements transportent des identifiants et versions ; un consommateur vérifie l'autorité avant de reconstruire sa projection. La compatibilité API peut préserver les routes existantes pendant que leur propriétaire change.

Domaine proposé	Plan de contrôle	Plan de données	Autorité / contrat
Gateway / contexte	Sessions et API compatibles ; contexte signé	Routage, quotas et limites de requête	Identité, projet, environnement, corrélation ; fail-closed
IAM / politique	Organisations, projets, membres, invitations, comptes et droits	Réconciliation IdP/OpenFGA, contrôle avant accès	Journal de commandes et politiques versionnées
Connections / egress	Connecteurs, secrets référencés, découverte et validation	Tests bornés et proxy sortant autorisé	Pas de secret dans manifeste, log ou édition
Flows	Manifests, graphes, versions, schedules, leases, cancellations	Placement et supervision de runs bornés	Commandes durables ; checkpoint lu dans édition publiée
Capture / journal	Configuration des protocoles et rétention	Capture durable mutualisée ; découpage de pages	Époque, partitions, ordre et preuve de durabilité
Trusted writer	Validation du plan, contexte et capacités	Écriture privée, fichiers immuables, candidat estampillé	PreparedWrite ; aucune capacité modify donnée au worker
Audits / publisher	Règles, approbations, préconditions et décisions	Audit du snapshot exact, publication et reçu	UUID, heads, ascendance, génération et preuve immuable
Catalog / lineage	Identité dataset, descriptions, labels, contrats, éditions	Indexation dérivée des publications autorisées	Découverte filtrée, provenance et pins
Serving / Icegres	Sessions, requêtes, profils de placement et quotas	Flight SQL, pgwire, Arrow, exécution native	ReadContext épinglé ; aucun droit implicite de publier
Files	Réservations, métadonnées, ACL et versions	Transfert, scan, hash, manifeste FILESET	URL limitée, objets privés avant publication

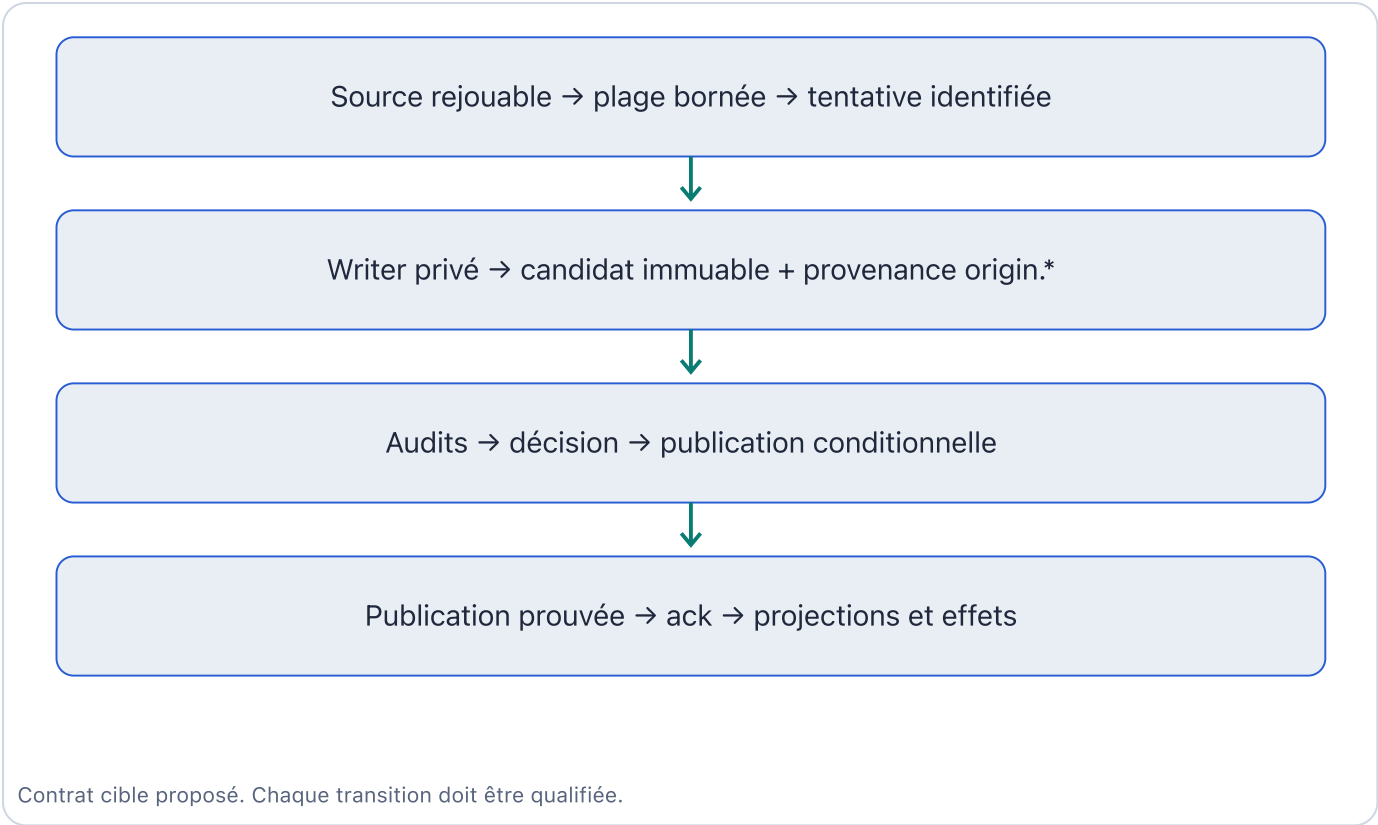
Domaine proposé	Plan de contrôle	Plan de données	Autorité / contrat
Eidos / semantic	Drafts, bundles, types métier et propositions	Compilation, projections, actions via Flows	Plan immuable, bundle et pins, aucune mutation main directe
Apps / devspaces	Spécifications, versions, routage, quotas et états désirés	Pods, sessions, logs et déploiements	Journal de contrôle indépendant du dataset
AI gateway / assistant	Conversations, budgets, outils, modèles et politiques	Appels inférence et outils sous identité déléguée	Consentement/approbation des effets et provenance des réponses
Embeddings / features	Spécifications, modèles et fraîcheur	Calcul et projection des éditions offline	Index et store online reconstruisibles, watermarks explicites
MCP / exposure	Outils publiés, schémas, permissions et consommateurs	Adaptateurs requête/recherche/action	Autorisation par ressource ; sources externes non fiables
Exports / delivery	Plan de livraison, format, destination et droits	Transfert d'une édition épinglée	Journal d'effets, clé idempotence et reçu externe
Notifications	Abonnements, templates, préférences et langues	Outbox, envoi et reprise des canaux	Déduplication et suppression d'informations privées
Operations / migration	Capacités, backups, migrations et rétention	Restore, compaction, jobs et purge planifiée	Plan approuvé, journal d'application et preuves de reprise

Références de l'analyse

[study](#) · [docs/hemera-v2/08-plateforme-rust-sans-jvm.html:50](#) Architecture consolidée, responsabilités et autorité des éditions.

Contrat commun de publication

- Les noms ci-dessous sont des propositions d'interface. Ils désignent des contenus obligatoires et doivent être versionnés dans les contrats partagés avant implémentation. Aucun champ isolé ne suffit à prouver une publication.
- La preuve d'audit est produite après le candidat et liée durablement à celui-ci. Elle ne peut pas être ajoutée rétroactivement par un simple fast-forward du snapshot. Le registre des publications et ses références retenues permettent de distinguer un snapshot approuvé d'un stage, y compris après une réponse réseau perdue.



Message	Contenu	Règle de validation
ReadContext	Sujet, tenant, projet, environnement, table UUID, snapshot ou release, politique	Résolu une fois, filtré puis transmis sans substitution silencieuse.
SourceRange	Source stable, époque, partitions, début/fin, transactions complètes	La plage couverte et le curseur publié restent cohérents après crash.
PreparedWrite	operation/attempt IDs, génération, table UUID, base heads, candidat, schéma/spec, summary origin.*, hashes	Les fichiers audités deviennent immuables pour le worker.
AuditDecision	Candidat exact, version des règles, résultats, approbateur, hash de preuve	Veto interdit la publication ; toute modification crée un nouveau candidat.
PublishReceipt	Publication autorisée, snapshot/release, candidat audité, opération, plage couverte	Trouvable après perte de réponse ; pas déduit du listing brut des snapshots.
AckReceipt	Système source, plage réellement acquittée, reçu de publication, statut observé	ACK_PENDING se réconcilie sans republier les données.
Release	Édition de manifeste, participants, pins membres, checkpoint commun	Lecteur voit la release ancienne ou nouvelle, sans mélange de membres.

Références de l'analyse

[study](#) · [docs/hemera-v2/09-decision-el4-wap.html:25](#) Frontière EL-4 et protocole writer/publisher.

Huit kinds, une autorité de publication

Kind	Contenu autoritaire	Contrat spécifique
TABLE	Données tabulaires	Snapshot, schéma, pins d'entrée et audits.
INCREMENTAL	Mise à jour bornée	Clés, stratégie de merge, plage source et idempotence.
STREAM	Raw rejouable publié par lots	Journal durable, ordre, rétention et audits avant ack.
VIEW	Définition publiée	Manifeste versionné et contrat de résolution des entrées.
FILESET	Ensemble d'objets	Manifeste Iceberg, objets immuables, hashes, ACL et rétention.
MODEL	Artefact de modèle	Manifeste, poids, signature, code, jeux d'entraînement et évaluations.
EMBEDDINGS	Vecteurs publiés	Modèle, inputs épinglés, vecteurs ; index dérivé.
FEATURESET	Features offline	Édition source ; store online dérivé avec watermark.

Références de l'analyse

[study](#) · [docs/hemera-v2/visuels/content.py:195](#) Taxonomie corrigée et distinctions EXPORT/EXPOSURE.

Manifeste, outils tiers et EL-4

- `source()` déclare le type de source, son identité stable, sa classe de durabilité, ses références de secrets, le contrat de schéma, la stratégie de curseur, la fenêtre bornée et ses audits. Les objets `SOURCE`, `OP` et `TEST` forment le graphe du flow ; `EXPORT` est un effet externe et `EXPOSURE` une déclaration de consommateur.
- `OLake` reste un extracteur `Go` encapsulé. Le `sidecar writer Java` ne fait pas partie de la cible. Si le `writer Rust` direct n'est pas qualifié pour une source, l'extraction aboutit à un journal privé puis à une édition `raw` audité. `STREAM` ne dispense jamais d'audit avant publication.
- `dbt`, `SQLMesh`, `dlt` et les workloads `Python` reçoivent des entrées épinglées et une sortie staging. Leur state interne peut accélérer l'exécution, mais la reprise source repart du checkpoint publié. Les scénarios de qualification incluent `bootstrap/CTID`, transactions multi-tables, `TRUNCATE`, absence de lignes mais avancement de curseur, slot partagé, crash et réponse de commit perdue.
- `DataFusion` est un core de calcul, pas une preuve de moteur distribué. Le placement interactif `Icegres`, les jobs `Python/DuckDB` et le calcul distribué ont des matrices distinctes. `Ballista` et `Sail` restent des candidats du dossier 08, à qualifier par version et workload ; leurs possibilités ne doivent pas être fusionnées en une promesse unique.

Références de l'analyse

[study](#) · [docs/hemera-v2/08-plateforme-rust-sans-jvm.html:159](#) Placement et limites des candidats de calcul.

[study](#) · [docs/hemera-v2/09-decision-el4-wap.html:25](#) Décision EL-4 sans `writer Java`.